

Submission Worksheet

Submission Data

Course: IT202-008-S2026

Assignment: IT202 Milestone 2

Student: George C. (gec23)

Status: Submitted | **Worksheet Progress:** 86%

Potential Grade: 8.35/10.00 (83.50%)

Received Grade: 0.00/10.00 (0.00%)

Started: 4/27/2026 1:01:20 PM

Updated: 5/4/2026 3:24:17 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone-2/grading/gec23>

View Link: <https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone-2/view/gec23>

Instructions

- Overview Link: https://youtu.be/t6X8dzZt_a0
- 1. Refer to Milestone2 of this doc:
<https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view>
- 2. Ensure you read all instructions and objectives before starting.
- 3. Ensure you've gone through each lesson related to this Milestone
- 4. Switch to the Milestone2 branch
 - 1. `git checkout Milestone2` (ensure proper starting branch)
 - 2. `git pull origin Milestone2` (ensure history is up to date)
- 5. Fill out the below worksheet
 - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
 - Ensure proper styling is applied to each page
 - Ensure there are no visible technical errors; only user-friendly messages are allowed
- 6. Once finished, click "Submit and Export"
- 7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 - 1. `git add .`
 - 2. `git commit -m "adding PDF"`
 - 3. `git push origin Milestone2`
 - 4. On Github merge the pull request from Milestone2 to qa
 - 5. On Github create a pull request from qa to prod and immediately merge. (This will trigger the prod deploy to make the Render.com prod links work)
- 8. Upload the same PDF to Canvas
- 9. Sync Local
 - 1. `git checkout qa`
 - 2. `git pull origin qa`

Section #1: (2 pts.) Data

Progress: 100%

≡ Task #1 (0.50 pts.) - API Data Table

Part 1:

Progress: 100%

Details:

- Show the table(s) structure of the table made to hold your API
- There should be at least 3 properties/fields beyond id, created, modified, and respective is_api or api_id columns
- Logical types and constraints should be set
- Show at least a few records from each API table (samples should include both API fetched data and custom entries noted by an api_id or is_api column)



data table



Saved: 5/1/2026 11:58:26 PM

Part 2:

Progress: 100%

Details:

- Add a direct link to your sql file from Github (should end in .sql)

URL #1

https://github.com/gcruzmedina/gec23-IT2023-008Milestone2/public_html/project/sql



URL

<https://github.com/gcruzmedina/>



Saved: 5/1/2026 11:58:26 PM

Part 3:

Progress: 100%

Details:

- Briefly note what each field/property represents

Your Response:

The table is designed to store information about various anime entries collected from an external API. Each record is uniquely identified by the id field, which serves as the primary key within the database, while the anime_id stores the corresponding identifier from the external API to prevent duplicate entries. The title field contains the name of the anime, and the type field indicates the format, such as TV series or movie. The status field describes whether the anime is currently airing, finished, or ongoing. Additionally, the episodes field records the total number of episodes available, and the score field represents the rating of the anime, typically on a decimal scale. Together, these fields provide a structured way to store, retrieve, and manage anime data efficiently within the application.

 Saved: 5/1/2026 11:58:26 PM

≡ Task #2 (0.50 pts.) - Data Transformation

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles the data transformation/conversion (from the API)
- Show a Render.com qa example of API getting fetched and loaded into the DB



one

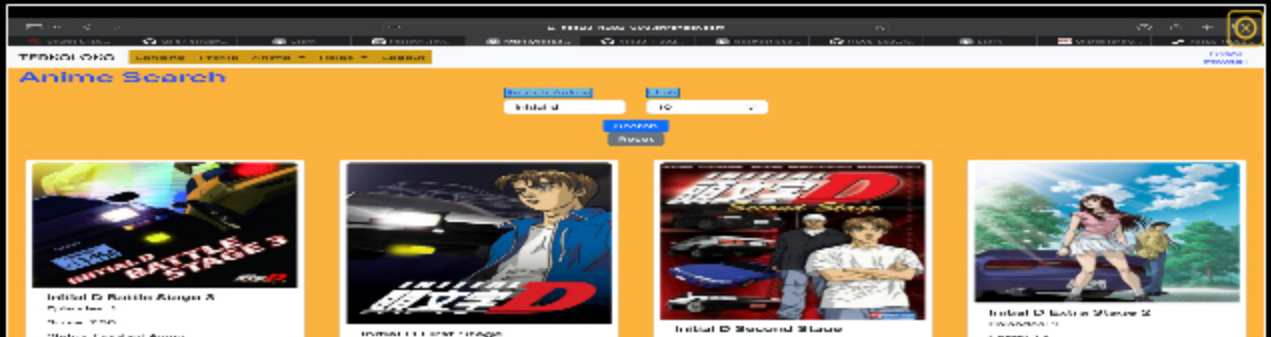


two





three



render

 Saved: 5/3/2026 10:05:29 PM

Part 2:

Progress: 100%

Details:

- Include Render.com prod link to API fetch page
- Include pull request link for this feature

URL #1

<https://gec23-it202-008.onrender.com/project/landing.php?anime=initial+d&limit=10>



URL
<https://gec23-it202-008.onrender.com/project/landing.php?anime=initial+d&limit=10>



URL #2

<https://github.com/gcruzmedina/gec23-it202-008>



URL
<https://github.com/gcruzmedina/gec23-it202-008>



 Saved: 5/3/2026 10:05:29 PM

Part 3:

Progress: 100%

Details:

- Will you be using all the data or just a subset?
- Do you need to convert any data or extract pieces of a property's value?
- Briefly explain the transformation/extraction logic to get the API data to the shape your application intends to use

Your Response:

The application only uses a subset of the API data, focusing on key fields like title, episodes, rating, and image. The JSON response is decoded and then processed by extracting and renaming certain fields (e.g., score to rating) to match the database structure. A loop is used to transform each API record into a clean format before inserting it into the database.



Saved: 5/3/2026 10:05:29 PM

≡ Task #3 (0.50 pts.) - API Duplicates

Progress: 100%

Details:

- Consider how duplicate entries are handled
 - What happens if you get the same result from the API for an entity that exists but has been unmodified?
 - What happens if you get the same result from the API for an entity that exists but has been modified?

Your Response:

The application prevents duplicates by checking if a record already exists before inserting. If the API returns an unchanged entity, no action is taken. If the entity exists but has updated values, the existing record is updated instead of inserting a duplicate. This ensures data consistency and avoids redundant entries.



Saved: 5/3/2026 10:06:22 PM

≡ Task #4 (0.50 pts.) - API Fetching

Progress: 100%

Details:

- Consider how your application will trigger this data fetch
 - (i.e., periodic via cron job, poor man's cron job, on app start/awake, etc)
 - (i.e., User or Admin triggered via a specific page/action)
 - (i.e., Lazy loaded during searches for stale/missing content)

Your Response:

The application primarily uses a user/admin-triggered approach to fetch API data, typically through a button or page action. This ensures controlled API usage and avoids unnecessary requests. Additionally, lazy loading can be used to fetch data on demand when missing or outdated. A cron-based automated refresh is not currently implemented but could be added for periodic updates.

Section #2: (1.5 pts.) Data Creation

Progress: 100%

Task #1 (60 pts.) - Validations

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the html validations (code)
- Show the javascript validations (code)
- Show the php validations (code)

```

<?php
// validate_email.php
function validate_email($email) {
    // Regular expression for email validation
    $regex = '/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/';
    // Check if the email matches the regex
    if (preg_match($regex, $email)) {
        return true;
    } else {
        return false;
    }
}

// validate_phone.php
function validate_phone($phone) {
    // Regular expression for phone validation
    $regex = '/^[0-9]{10}$/';
    // Check if the phone number matches the regex
    if (preg_match($regex, $phone)) {
        return true;
    } else {
        return false;
    }
}
    
```

one

```

<?php
// validate_email.php
function validate_email($email) {
    // Regular expression for email validation
    $regex = '/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/';
    // Check if the email matches the regex
    if (preg_match($regex, $email)) {
        return true;
    } else {
        return false;
    }
}

// validate_phone.php
function validate_phone($phone) {
    // Regular expression for phone validation
    $regex = '/^[0-9]{10}$/';
    // Check if the phone number matches the regex
    if (preg_match($regex, $phone)) {
        return true;
    } else {
        return false;
    }
}
    
```

two

```

<?php
// validate_email.php
function validate_email($email) {
    // Regular expression for email validation
    $regex = '/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/';
    // Check if the email matches the regex
    if (preg_match($regex, $email)) {
        return true;
    } else {
        return false;
    }
}

// validate_phone.php
function validate_phone($phone) {
    // Regular expression for phone validation
    $regex = '/^[0-9]{10}$/';
    // Check if the phone number matches the regex
    if (preg_match($regex, $phone)) {
        return true;
    } else {
        return false;
    }
}
    
```

three

```
function validateForm() {  
    // Validate required fields  
    if (document.getElementById("name").value === "" ||  
        document.getElementById("email").value === "" ||  
        document.getElementById("password").value === "" ||  
        document.getElementById("confirm_password").value === "") {  
        alert("All fields are required.");  
        return false;  
    }  
    // Validate email format  
    const emailRegex = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;  
    if (!emailRegex.test(document.getElementById("email").value)) {  
        alert("Invalid email format.");  
        return false;  
    }  
    // Validate password strength  
    const password = document.getElementById("password").value;  
    if (password.length < 8 || !/[0-9]/.test(password) || !/[a-zA-Z]/.test(password)) {  
        alert("Password must be at least 8 characters long, containing at least one number and one letter.");  
        return false;  
    }  
    // Validate confirm password  
    if (password !== document.getElementById("confirm_password").value) {  
        alert("Passwords do not match.");  
        return false;  
    }  
    // If all validations pass, submit the form  
    document.getElementById("form").submit();  
}
```

four

Saved: 5/3/2026 10:18:10 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the html validations used
- Briefly explain the javascript validation logic
- Briefly explain the php validation logic
- Note which role(s) have access to this create form

Your Response:

The application uses multiple layers of validation to ensure data integrity. HTML validation is implemented using built-in input attributes such as required, min, max, and maxlength, which enforce that users provide values within acceptable ranges before submission. JavaScript validation adds another layer by checking inputs when the form is submitted, ensuring conditions like minimum text length and valid numeric ranges are met; if not, the submission is prevented using `event.preventDefault()` and an error message is shown. PHP validation is performed on the server side to verify all submitted data before it is processed or stored, including checks for empty values, correct data types, and valid ranges, ensuring security since client-side validation can be bypassed. Access to the create form is restricted to authorized users, typically those with the Admin role, so only permitted users can create new records in the system.

Saved: 5/3/2026 10:18:10 PM

Task #2 (60 pts.) - Examples

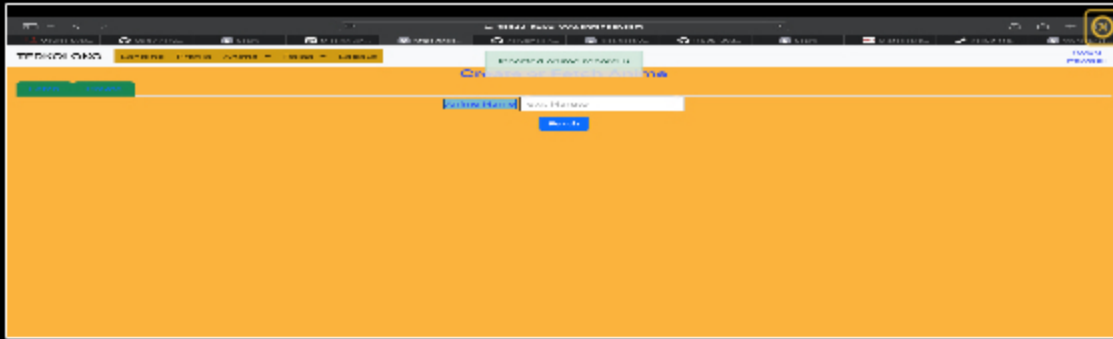
Progress: 100%

Details:

- Show a successful creation
- Show how a duplicate submission is handled (i.e., what message or behavior

occurs)

- Demonstrate various validation errors
- Ensure Render.com qa url is visible



create



table



repeat/error



Saved: 5/4/2026 12:48:44 PM

Task #3 (30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL #1

[https://gec23-](https://gec23-it202-008.onrender.com/project/admin/create_anime.php)

[it202-008.onrender.com/project/admin/create_anime.php#](https://gec23-it202-008.onrender.com/project/admin/create_anime.php)



URL

<https://gec23-it202-008.onrender.com>



URL #2

<https://github.com/gcruzmedina/gec23-IT202-008/pull/54>



URL

<https://github.com/gcruzmedina/gec23-IT202-008/pull/54>



Saved: 5/4/2026 12:49:22 PM

Section #3: (1.5 pts.) Data List Page (Many Items)

Progress: 100%

Task #1 (60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles the filter/sort logic
- Show the code that enforces the record limit (valid range 1-100, default 10 if out of range)
- Show the code that generates the list/grid output
- Styling must be applied



one



two



Saved: 5/4/2026 1:58:30 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain how the filter/sort logic works and formulates the results
- Briefly explain how the limit is validated and applied server-side
- Briefly explain how the list/grid output is generated
- Briefly note the styling choices

Your Response:

The application implements filtering and sorting by reading user input from query parameters (such as search terms or sort options) and dynamically building the SQL query. Filtering is typically done using a LIKE clause to match partial titles, while sorting is applied using an ORDER BY clause with only allowed fields to ensure valid input. The record limit is validated server-side by checking that the provided value is numeric and within an acceptable range (1–100); if it is missing or invalid, a default value of 10 is used. This validated limit is then applied to the query using the LIMIT clause to control how many records are returned. The results are retrieved using a prepared statement and displayed through a loop that generates HTML elements for each record, forming a list or grid layout. Styling is applied using CSS classes to organize the output into structured cards or rows, improving readability and providing a clean, responsive visual layout.



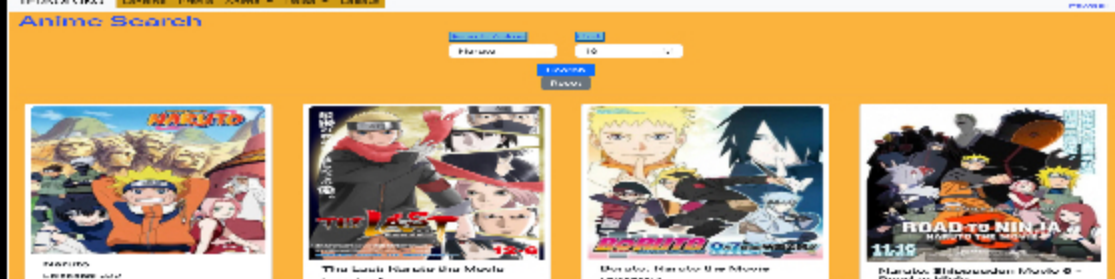
Saved: 5/4/2026 1:58:30 PM

Task #2 (60 pts.) - Examples

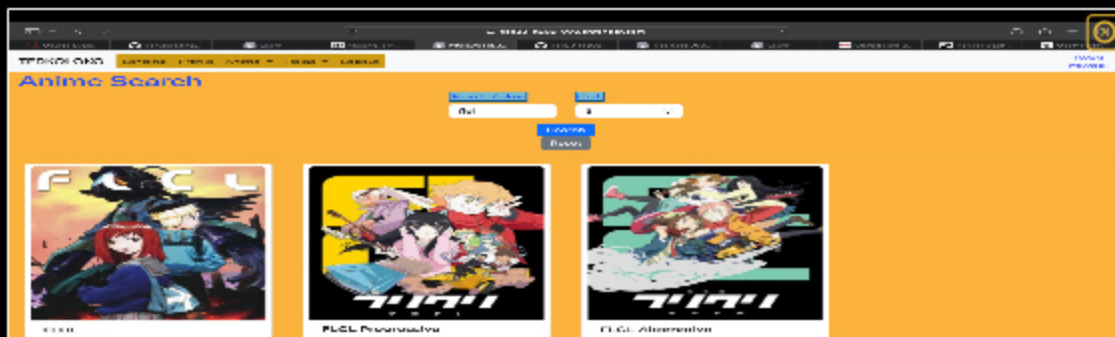
Progress: 100%

Details:

- Show a few variations of filter/sort applied (including no records - should display a "no results" message)
- Show both API-fetched and manually created entities present in the list
- Show the limit field in use with at least two different values
- Each list item should be a summary
- Each list item should have the following links
 - A link to a single view of the specific entity (i.e., a details page)
 - A link to delete this entity (this may be an admin-only functionality, but it should be present for the respective role)
 - A link to edit this entity (this may be an admin-only functionality, but it should be present for the respective role)
- Ensure Render.com qa url is visible



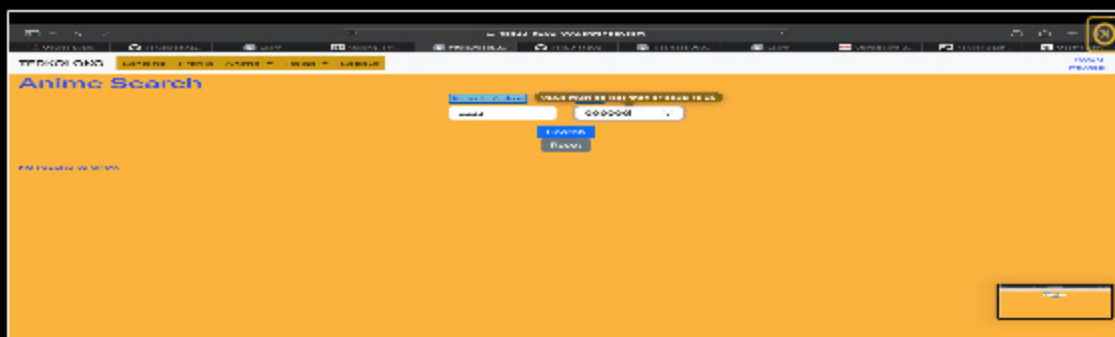
one



two



three



four



Saved: 5/4/2026 2:03:12 PM

Task #3 (30 pts.) - Links

Progress: 100%

Details:

Details:

- Briefly explain how the loading logic works
- Briefly explain how the entity output is generated
- Briefly note the styling choices

Your Response:

The loading logic begins by retrieving the id from the URL using `$_GET`, converting it to an integer to ensure validity, and checking if it is missing or invalid; if so, the user is redirected back with an error message using `flash()`. If the id is valid, the application connects to the database via `getDB()` and executes a prepared SQL statement to safely fetch the corresponding record, preventing SQL injection, and if no record is found, it redirects again with a warning. Once the record is successfully loaded, the entity output is generated by embedding the data into HTML using PHP `echo` statements, with `htmlspecialchars()` applied to each field to prevent XSS attacks and ensure safe rendering. The content is displayed in a structured card layout for clarity, and styling is handled using Bootstrap classes already included in the project (such as `container`, `card`, `shadow`, and `btn` classes), providing a clean, responsive design without requiring custom CSS.



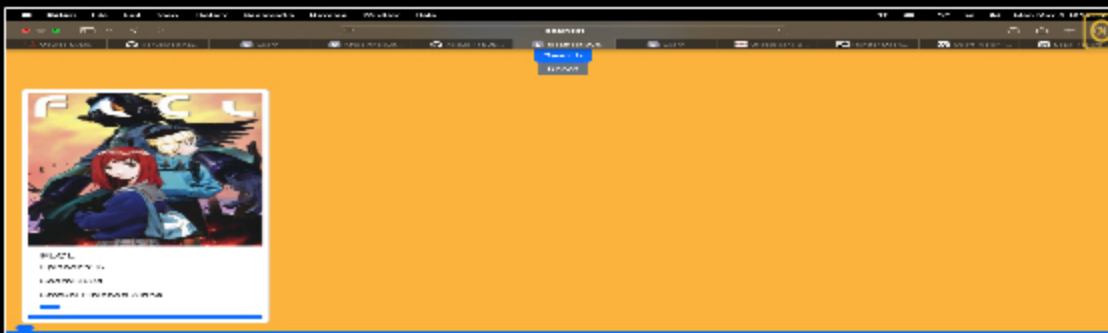
Saved: 5/4/2026 2:40:42 PM

Task #2 (60 pts.) - Examples

Progress: 75%

Details:

- Show a few examples of different entities
- Each entity item should have more details than the summary view (if possible)
- Each entity should have the following links
 - A link to edit the single entity (this may be an admin-only thing, but it should be present for the respective role)
 - A link to delete the single entity (this may be an admin-only thing, but it should be present for the respective role)



Missing Caption

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

- Details:**

 - Include the Render.com prod link to this page
 - Include pull request link for this feature

URL #1
<https://gec23-it202-008.onrender.com/project/landing.php?anime=zzzz&limit=25>

URL
<https://github.com/gcruzmedina/gec23-IT202-008/pull/54>

URL #1
<https://gec23-it202-008.onrender.com/project/landing.php?anime=zzzz&limit=25>

URL
<https://github.com/gcruzmedina/gec23-IT202-008/pull/54>

URL #2 <https://github.com/gcruzmedina/gec23-IT202-008/pull/54>     

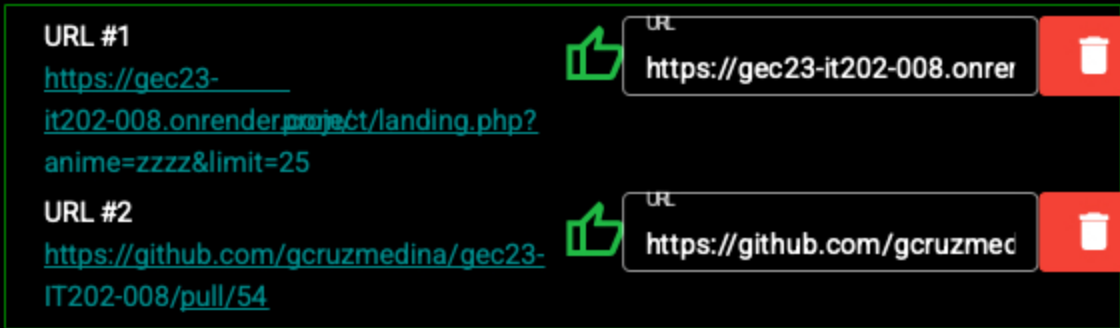
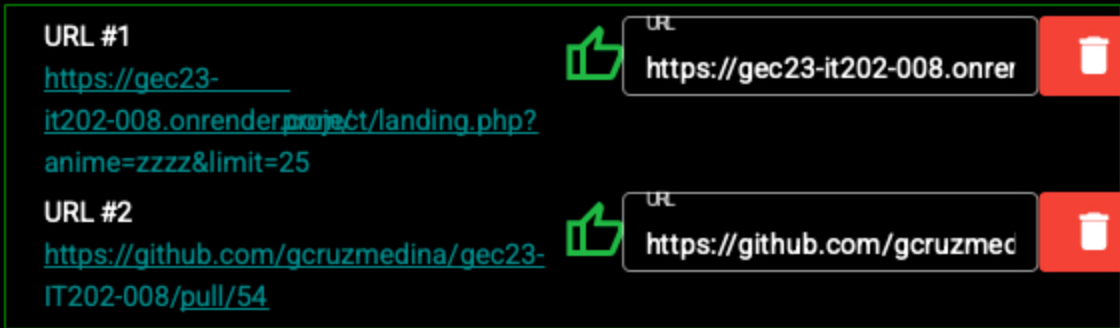
URL #2 <https://github.com/gcruzmedina/gec23-IT202-008/pull/54>     

URL #1
<https://gec23-it202-008.onrender.com/project/landing.php?anime=zzzz&limit=25>

URL
<https://github.com/gcruzmedina/gec23-IT202-008/pull/54>

URL #1
<https://gec23-it202-008.onrender.com/project/landing.php?anime=zzzz&limit=25>

URL
<https://github.com/gcruzmedina/gec23-IT202-008/pull/54>



URL #2 <https://github.com/gcruzmedina/gec23-IT202-008/pull/54>     

URL #2 <https://github.com/gcruzmedina/gec23-IT202-008/pull/54>     



 Saved: 5/4/2026 2:41:17 PM

Section #5: (1.5 pts.) Edit Page

Progress: 33%

≡ Task #1 (60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

- Show the code that handles loading of the record (and how the id retrieved for usage)
 - Include the code that handles invalid/missing ids
- Show the code that generates the form with the correct data types and populates it
- Show the html, javascript, and php validation (should be similar to create)
- Styling must be applied

- Show the code that handles loading of the record (and how the id retrieved for usage)
 - Include the code that handles invalid/missing ids
- Show the code that generates the form with the correct data types and populates it
- Show the html, javascript, and php validation (should be similar to create)
- Styling must be applied



Saved: 5/4/2026 3:16:10 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the loading logic works
- Briefly explain how the form is generated and submission is handled
- Briefly note the styling choices
- Note what fields are editable and why the choice was made for them (it's likely not all fields are editable, especially `id`, `created`, `modified`, etc)

Your Response:

The loading logic begins by retrieving the `id` from the URL using `$_GET`, validating that it is a positive number, and redirecting with a flash message if it is missing or invalid; if valid, the application connects to the database using `getDB()` and executes a prepared statement to safely fetch the matching record, redirecting again if no record is found. The form is generated using a structured array of field definitions (passed into helper functions like `render_input()`), which dynamically builds the inputs and pre-fills them with existing values from the loaded record; when the form is submitted (typically via POST), the server validates the input, sanitizes it, and runs an UPDATE query to persist changes to the database, followed by a redirect and success/error message. Styling is handled באמצעות Bootstrap classes already included in the project (such as `container`, `row`, `card`, `form-control`, and `btn`), providing a consistent and responsive layout without custom CSS. Editable fields generally include user-facing data like `title`, `episodes`, `score`, `status`, or `synopsis` because these are meaningful to update, while fields like `id`, `created`, and `modified` are not editable since they are system-managed identifiers and timestamps that ensure data integrity and proper record tracking.



Saved: 5/4/2026 3:16:10 PM

Task #2 (60 pts.) - Examples

Progress: 0%

Details:

- Show a before and after of updating an entity (clearly caption)
- Successful update should have an appropriate message shown
- The updated data should be shown in the form
- Any errors should have user-friendly messages shown
- Show some examples of validation errors (html, javascript, php)





Missing Caption

 Not saved yet

Task #3 (30 pts.) - Links

Progress: 0%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL #1

Missing URL

URL

 Not saved yet

Section #6: (1.5 pts.) Delete Logic

Progress: 66%

Task #1 (60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles deletion of the record (and how the id retrieved for usage)
 - Include the code that handles invalid/missing ids (should redirect to the list page with an appropriate message)
- Handle any necessary roles/permissions (i.e., it's not likely that anyone can delete any entity they choose)
- Show the redirect logic that goes back to the previous page (where the route came from), if it was a list page, it should maintain the filter/sort criteria





code



Saved: 5/4/2026 3:24:17 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the deletion logic works
- Is it a soft or hard delete?
- Note any permissions/rules applied.
- Briefly explain how the redirect maintains the previous route and any filter/sort data

Your Response:

The deletion works by getting the id from the URL, making sure it's valid, and checking that the user has permission (like being an Admin). If everything is okay, the code runs a SQL DELETE query to remove the record from the database, which is a hard delete since the data is permanently removed. If the id is missing or invalid, the user is redirected back to the list page with an error message. After a successful delete, the page redirects back to where the user came from, using parameters in the URL to keep any search or filter values so the page looks the same as before.



Saved: 5/4/2026 3:24:17 PM

Task #2 (60 pts.) - Examples

Progress: 0%

Details:

- Show a successful delete message
- Show a before and after database view of the record being deleted (clearly caption)
- Any errors should have user-friendly messages shown





Missing Caption



Not saved yet

Task #3 (30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page (even though it's not a full page, it's more like how logout works)
- Include pull request link for this feature

URL #1

<https://gec23-it202-008.onrender.com/project/landing.php?anime=naruto&limit=10>



URL

<https://gec23-it202-008.onrender.com/project/landing.php?anime=naruto&limit=10>



Saved: 5/4/2026 3:16:55 PM

Section #7: (0.5 pts.) Misc

Progress: 100%

Task #1 (16.67 pts.) - Github Details

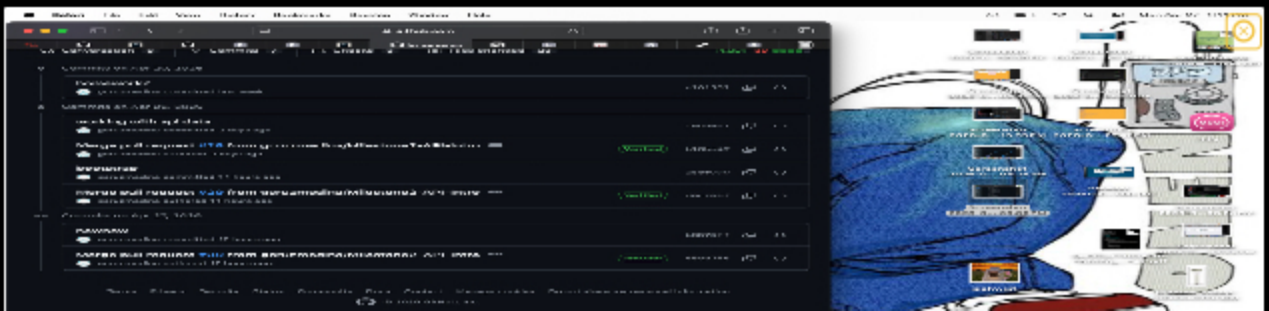
Progress: 100%

Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history (it may not be possible to capture all, just grab a reasonable chunk)



commits

 Saved: 4/27/2026 1:12:03 PM

Part 2:

Progress: 100%

Details:

Include the link to the Pull Request (should end in /pull/#) (Milestone2 into qa)

URL #1

<https://github.com/gcruzmedina/gec23-IT2023-0031>



URL

[https://github.com/gcruzmedina/!](https://github.com/gcruzmedina/)

 Saved: 4/27/2026 1:12:03 PM

Task #2 (16.67 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click Projects and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



one





two



Saved: 4/27/2026 1:09:55 PM

≡ Task #3 (16.67 pts.) - Reflection

Progress: 100%

⇒ Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I learned how implement bootstrap into my website and was also able to make adjustments to the landing page.



Saved: 4/27/2026 1:08:03 PM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

Honestly none of it, i was lowkey fighting for my life in order to get this stuff done, but then again i am not a coder so all of this just looks hard to me fr.



Saved: 4/27/2026 1:07:09 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

getting it to work in render was a bit of an issue, but once everything was smoothed out, it was real chill. It was a bit different from the last module, but there was some useful stuff to learn.



Saved: 4/27/2026 1:06:37 PM